
Student Record Management System

Program Design Report

Abigaba Patience Sarah

24/U/0071

1. Program Design

The system is a menu-driven Python terminal application for managing student records across two persistent files: `students.csv` (core academic data with Reg No, Name, Gender, Age, Course, Score) and `students.json` (extended details with address, contact, program, date added). A third file, `student_system.log`, records every user action and system error with timestamps. The application presents a numbered menu (Add, View, Search, Update, Delete, Exit) and loops until the user chooses to exit.

2. Key Functions

- `load_csv() / save_csv()` : reads and writes student records using `csv.DictReader` and `csv.DictWriter`.
- `load_json() / save_json()` : reads and writes extended details using `json.load()` and `json.dump()`.
- `add_student()` : validates all fields, checks for duplicates, then appends to both CSV and JSON.
- `view_all_students()` : loads and prints all records in a formatted table with column headers.
- `search_student()` : finds a student by Reg No and displays both CSV and JSON details.
- `update_student()` : loads current values, allows field-by-field edits (Enter = keep), then saves.
- `delete_student()` : confirms with user before removing the record from both CSV and JSON.

3. Exception Handling Strategy

Every function is wrapped in a `try/except/finally` block. Four custom exception classes inherit from `Exception`:

- `StudentNotFoundError` : raised in search, update, and delete when a Reg No does not exist.
- `DuplicateRegistrationError` : raised in `add_student()` when the Reg No is already in the system.
- `InvalidScoreError` : raised when score is not a number between 0 and 100.
- `InvalidAgeError` : raised when age is not a whole number between 18 and 60.

Built-in exceptions handled include `ValueError` (empty fields, bad gender), `json.JSONDecodeError` (corrupt JSON), and `KeyboardInterrupt` (Ctrl+C exits gracefully). The `finally` block in each function always prints a closing separator, confirming the action has ended whether it succeeded or failed.

4. Testing Results

The following test cases were run to verify all operations and error paths:

	Test Case	Expected Result	Result
1	Add student with valid data	Saved to CSV and JSON	Pass
2	Add duplicate Reg No	DuplicateRegistrationError raised	Pass
3	Add student, score = 150	InvalidScoreError raised	Pass
4	Add student, age = 8	InvalidAgeError raised	Pass
5	View all students	Formatted table displayed	Pass
6	Search existing Reg No	CSV + JSON details printed	Pass

7	Search non-existent Reg No	StudentNotFoundError raised	Pass
8	Update with invalid score	InvalidScoreError, no save	Pass
9	Delete, confirm = 'no'	Record kept, cancelled message	Pass
10	Delete existing student	Removed from CSV and JSON	Pass
11	Invalid menu choice	Warning printed, menu loops	Pass

All 11 test cases passed. The log file correctly captured every action with appropriate INFO, WARNING, and ERROR levels.